

OSSP-PRACTICAL-WEEK2

Task1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user by accepting a Linux command as input, creating a child process using the fork() system call, executing the command in the child process using an appropriate exec() system call, allowing the parent process to wait for the child using wait(), and displaying the Process ID (PID) of both the parent and child processes.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    char command[100];
    char *args[20];
    char *token;
    int i = 0;

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline
    command[strlen(command)] = '\0';

    // Split command into arguments
    token = strtok(command, " ");

    while (token != NULL && i < 19) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command...\n");
        execvp(args[0], args);
    }
}
```

```
        execvp(args[0], args);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}
```

Output:

```
root@LAPTOP-1C0JKKG6:~# nano week2pt1.c
root@LAPTOP-1C0JKKG6:~# gcc week2pt1.c -o week2pt1
root@LAPTOP-1C0JKKG6:~# ./week2pt1
Enter a Linux command: ls

Parent Process
Parent PID : 520
Child PID : 521
Parent waiting for child...

Child Process
Child PID : 521
Parent PID : 520
Executing command: ls

week2pt1 week2pt1.c week3pt1 week3pt1.c week3pt2 week3pt2.c

Child process completed.
root@LAPTOP-1C0JKKG6:~# |
```

Observation: I learned how the Linux operating system creates a child process using fork() and executes a Linux command using the execvp() system call. I also understood how the parent process waits for the child process using wait() and how PID and PPID help identify the relationship between parent and child processes.

Task2: Using Linux terminal commands such as uname, lscpu, lsblk, ps, and top, investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the operating system abstracts and manages CPU, memory, storage, and I/O devices.

Output:

Using command `uname -a`:

```
root@LAPTOP-1C0JKKG6:~# uname -a
Linux LAPTOP-1C0JKKG6 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@LAPTOP-1C0JKKG6:~# |
```

Using command `lscpu`:

```
root@LAPTOP-1C0JKKG6:~# lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       12
On-line CPU(s) list:         0-11
Vendor ID:                    GenuineIntel
Model name:                   Intel(R) Core(TM) 5 120U
CPU family:                   6
Model:                        186
Thread(s) per core:           2
Core(s) per socket:           6
Socket(s):                    1
Stepping:                     3
BogoMIPS:                     4992.00
Flags:                        fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx
                             pdpe1gb rdtscp lm constant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmulqdq
                             vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor l
                             ahf_lm abm 3dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjust bmi1 avx
                             2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves avx_vnni vnni umip w
                             aiatpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capabilities

Virtualization features:
  Virtualization:              VT-x
  Hypervisor vendor:            Microsoft
  Virtualization type:          full
Caches (sum of all):
  L1d:                          288 KiB (6 instances)
  L1i:                          192 KiB (6 instances)
  L2:                           7.5 MiB (6 instances)
  L3:                           12 MiB (1 instance)
NUMA:
  NUMA node(s):                 1
  NUMA node0 CPU(s):            0-11
Vulnerabilities:
  Gather data sampling:         Not affected
  Ghostwrite:                   Not affected
  Indirect target selection:    Not affected
  Itlb multihit:                Not affected
  L1tf:                          Not affected
  Mds:                           Not affected
  Meltdown:                     Not affected
  Mmio stale data:              Not affected
  Old microcode:                Not affected
  Reg file data sampling:       Vulnerable: No microcode
  Retbleed:                     Mitigation; Enhanced IBRS
  Spec rstack overflow:         Not affected
  Spec store bypass:            Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:                   Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:                   Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW sequence; BHI BHI_DIS_S
  Srbds:                         Not affected
  Tsa:                           Not affected
  Tsx async abort:              Not affected
  Vmscape:                       Not affected
root@LAPTOP-1C0JKKG6:~# |
```

Using command `lsblk`:

```
root@LAPTOP-1C0JKKG6:~# lsblk
NAME
MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.4M  1 disk
sdc   8:32   0    2G    0 disk [SWAP]
sdd   8:48   0    1T    0 disk /mnt/wslg/distro
/
```

Using command `ps`:

```

root@LAPTOP-1COJKKG6:~# ps
  PID TTY          TIME CMD
  296 pts/0    00:00:00 bash
  600 pts/0    00:00:00 ps
root@LAPTOP-1COJKKG6:~# |

```

Using command top:

```

root@LAPTOP-1COJKKG6:~# top
top - 05:32:19 up 55 min,  1 user,  load average: 0.04, 0.04, 0.00
Tasks:  22 total,   1 running, 21 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  7782.6 total,  7168.4 free,   532.8 used,   231.9 buff/cache
MiB Swap: 2048.0 total,  2048.0 free,    0.0 used,  7249.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   21704   13056   9736 S   0.0   0.2   0:00.52 systemd
    2 root        20   0    3180    2208   2072 S   0.0   0.0   0:00.01 init-systemd(Ub
    7 root        20   0    3196    2128   2008 S   0.0   0.0   0:00.00 init
   53 root        19  -1   34048   12692   11620 S   0.0   0.2   0:00.28 systemd-journal
   98 root        20   0   25004    6744   5184 S   0.0   0.1   0:00.54 systemd-udev
  109 systemd+    20   0   21344   13140   11012 S   0.0   0.2   0:00.09 systemd-resolve
  110 systemd+    20   0   91036   7948    6980 S   0.0   0.1   0:00.09 systemd-timesyn
  170 root        20   0    4244    2684   2432 S   0.0   0.0   0:00.06 cron
  171 message+    20   0    9532    5368   4672 S   0.0   0.1   0:00.07 dbus-daemon
  181 root        20   0   17984    8836   7792 S   0.0   0.1   0:00.09 systemd-logind
  184 root        20   0 1829836   11904   9460 S   0.0   0.1   0:00.21 wsl-pro-service
  192 root        20   0    3124    1972    1836 S   0.0   0.0   0:00.00 agetty
  194 syslog      20   0  222516    6088   4668 S   0.0   0.1   0:00.08 rsyslogd
  210 root        20   0  107032   23316   13860 S   0.0   0.3   0:00.10 unattended-upgr
  294 root        20   0    3184    1112    980 S   0.0   0.0   0:00.00 SessionLeader
  295 root        20   0    3200    1248   1108 S   0.0   0.0   0:00.36 Relay(296)
  296 root        20   0    6080    5324    3648 S   0.0   0.1   0:00.12 bash
  297 root        20   0    6076    4512    3740 S   0.0   0.1   0:00.01 login
  340 root        20   0   20320   11488    9360 S   0.0   0.1   0:00.04 systemd
  341 root        20   0   21168   3516    1804 S   0.0   0.0   0:00.00 (sd-pam)
  361 root        20   0   5948    5264    3648 S   0.0   0.1   0:00.00 bash
  604 root        20   0    9332    5716   3500 R   0.0   0.1   0:00.02 top

```

Observation: I learned how to use Linux commands to examine system hardware and running processes. I understood how the operating system manages CPU, storage, memory, and processes and provides these resources to applications through system services